

Parsek — Logistics (Supply Routes) Design

Design specification for Parsek's stock-first automated cargo delivery system — covering Supply Run detection, Supply Route creation, dispatch scheduling, resource/inventory transfer between vessels, endpoint resolution, transfer window computation, and future round-trip linking across the rewind timeline.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies how committed recording chains can be confirmed as Supply Runs, then turned into Supply Routes that repeat the stock resource and inventory transfers the player already performed.

Version: 0.3 (renamed to Logistics / Supply Routes and tightened around stock-first Supply Runs) **Prerequisite:** Phase 10 (Location Context) is complete and Phase 11 provides the base resource/inventory manifests. Logistics v1 also requires the connection-scoped capture extensions listed in section 4.9 and tracked in `roadmap.md` under Phase 13 prerequisites. **Out of scope:** Ghost playback engine, recording system, chain structure, game actions recalculation engine. See `parsek-flight-recorder-design.md` and `parsek-game-actions-and-resources-recorder-design.md` for those.

1. Introduction

This document specifies how Parsek turns a player-flown Supply Run into an automated Supply Route. A Supply Route is chain-sequential: the player flies a cargo mission, docks to a destination, uses stock KSP transfer systems, undocks, commits the recording, confirms that the run should become a route, and Parsek replays the source chain on each dispatch cycle while repeating the recorded cargo transfer.

- Logistics window / Supply Routes vocabulary
- Supply Run detection and route analysis
- How stops and delivery manifests are derived from docking windows in the committed chain
- Chain-sequential playback: source recordings play in order, delivery between recordings, restart after dispatch interval
- The dispatch cycle: check timing, check capacity, charge or deduct origin cargo, deliver
- Endpoint resolution: surface proximity fallback vs orbital PID matching
- Transfer window scheduling via synodic period computation
- Future round-trip linking for paired one-way routes
- Cargo modification on unloaded vessels (resources via `ProtoPartResourceSnapshot`, inventory via `ModuleInventoryPart`)
- Timeline integration with epoch isolation for revert safety
- Module architecture: self-contained `Logistics/` directory with 4 integration seams
- Edge cases: destruction, full tanks, competing routes, time warp, reverts

1.1 What happens when the player creates a route

The player flies a mission normally — drive a rover with fuel to a base, launch a tanker to an orbital station, or send a cargo lander to a depot. During the flight, the transport docks to the destination, transfers cargo through stock KSP systems, then undocks. Docking ports are the only v1 connection type. The route model still names the broader concept "stock connection window" so claw/grapple or stock crossfeed/fuel-line support can be added later without replacing the route shape, but v1 should not block on those paths.

After the recording commits, Parsek analyzes the committed chain. If it finds a complete docking connection window with a delivery resource or inventory delta, the merge/recordings UI prompts: **"Create Supply Route from this Supply Run?"** The player sees a route summary (origin, endpoint, delivery manifest, KSC dispatch cost if any, transit time), sets or accepts the dispatch interval, and confirms. The Supply Route goes live.

No explicit **Record Supply Run** button is required for v1. Reducing actions is the better default: if the player docked, moved cargo, undocked, and committed, Parsek should assume route creation is likely and ask. A later helper button may still mark intent or suppress unrelated prompts, but it must not be required for correctness.

1.2 What happens each cycle

On schedule, the route scheduler begins a chain-sequential cycle. It tells the ghost playback engine to play the first source recording when visuals are available, but route state advances by UT. When the recorded delivery boundary UT is reached, delivery happens at the endpoint. The scheduler continues through the remaining source recordings, then waits for the dispatch interval before starting the next cycle.

Before each cycle, the route evaluates whether dispatch is possible. It checks the destination (is there room?), the origin (are there enough resources or inventory items, unless it is KSC), the KSC funds cost in Career, and whether the transit window is open. If everything checks out, origin cargo is deducted or the KSC dispatch funds cost is charged. The ghost replays the recorded chain visually during transit.

1.3 What the player sees

Situation	What happens
Eligible Supply Run committed	Parsek offers to create a Supply Route from the run.
Supply Route confirmed	Route analysis extracts the endpoint and cargo manifest from the committed chain. Player sees summary, sets interval, confirms.
Route dispatches on schedule	Proven non-KSC origin cargo is deducted, or KSC dispatch cost is charged in Career. Ghost begins chain-sequential replay.
Route delivers	Cargo (resources, inventory) appears at the resolved endpoint vessel as the ghost reaches the recorded transfer point.
Destination tanks full	Cycle skipped. Origin NOT deducted. No ghost replay for skipped cycle.
Origin runs out of resources	Dispatch delayed until resources available. Route resumes automatically.
Destination destroyed (surface)	Nearest-compatible vessel fallback can reconnect to a rebuilt base at the same location.
Destination destroyed (orbital)	Route halted (<code>EndpointLost</code>). Player must re-target to new station.
Source recording missing	Route halted (<code>MissingSourceRecording</code>). No cargo transfers without the proof recording.
Source recording deleted during transit	Current cycle aborts before delivery and route becomes <code>MissingSourceRecording</code> .
Player reverts past a dispatch	Epoch isolation invalidates dispatch. Stock save rollback or ledger recomputation restores cargo/funds.
Two routes linked as round-trip	Future feature: they alternate, Route A completes -> Route B dispatches -> B completes -> A dispatches.

1.4 Example: fuel delivery rover

SUPPLY RUN:

Phase 1: Rover departs KSC runway with 200 LF
Phase 2: Rover docks at base (connection EndResources: 195 LF)
Phase 3: Player transfers 150 LF to base (rover: 45 LF, base: +150 LF)
Phase 4: Rover undocks and drives clear
Player commits recording; Parsek offers "Create Supply Route?"

ROUTE ANALYSIS PRODUCES:

Origin: KSC runway area (Career dispatch cost = stock part cost + used/delivered cargo cost)
Endpoint: base location (from stock connection target PID + connection coordinates)
Delivery: 150 LF per cycle (rover loses 150 LF; base gains 150 LF and retains it through undock)
Transit: chain duration (~10 min)
Interval: player-set (minimum = chain duration)

EACH CYCLE (chain-sequential):

UT=0: Dispatch. Career funds charged if applicable. Ghost rover starts the first source recording.
UT=stop: Recorded delivery boundary reached. 150 LF added to base, clamped to tank capacity.
Ghost continues through the rest of the source chain.
UT=total: Chain complete. Cycle done. Wait for dispatch interval.
If base full at dispatch check -> 0 LF added, origin not deducted.

1.5 Example: Minmus mining supply chain

ROUTE A: KSC -> Minmus Base (fuel supply)

Origin: KSC (Career dispatch cost charged)
Delivery: 800 LF + 978 Ox per cycle
Result: mining base stays fueled for drill + ISRU operation

ROUTE B: Minmus Base -> Kerbin Depot (ore delivery)

Origin: Minmus base (non-KSC -- Supply Run starts docked to the base depot)
Cost: 1200 Ore + 600 LF + 733 Ox per cycle (used/delivered manifest)
Delivery: 1200 Ore per cycle
Gate: dispatch delayed until base has all required resources

CHAIN BEHAVIOR:

Route A delivers fuel -> base mines ore -> Route B ships ore to depot.
If Route A stops -> base runs dry -> Route B pauses indefinitely.
If depot tanks full -> Route B skips cycles, base accumulates ore.

2. Design Philosophy

These principles govern every design decision in the logistics system. They are listed here because they inform every section that follows.

2.1 Realism and fidelity

1. **Fly it once, automate it forever.** Routes replicate exactly what the player did during the recording. The delivery amount, transit duration, and fuel cost are all derived from the real flight — not configured abstractly.
2. **Transit takes real time.** The route duration equals the recording duration. A 3-year Eeloo transfer takes 3 years per cycle. A 10-minute rover drive takes 10 minutes. No shortcuts.
3. **Transfer windows are respected.** Inter-body routes dispatch at the synodic period of the origin and destination bodies, phase-anchored to the original Supply Run start UT. Same-body routes can dispatch at any time but not faster than the original recording.

2.2 Resource safety

4. **Stock proof-of-work.** Parsek may automate stock actions the player already performed, but it must not invent storage, cargo, transfer rules, crew, or production rules. A Supply Route exists because a committed Supply Run proves the transport, path, stock connection, cargo delta, and disconnect. In v1 that proof is specifically dock, deliver, and undock.
5. **No infinite cargo glitches.** Routes deliver exactly what was transferred during the recording — no more, no less. KSC origins are not free in Career: each dispatch charges stock-realistic funds for the source vessel parts plus the resource/inventory quantities used or delivered by the Supply Run. Non-KSC origins deduct the resource/inventory quantities used or delivered from a real origin vessel, but v1 only allows that when the run starts docked to that origin depot and records its PID. Recovery funds from the original flown vessel are one-time unless a later round-trip design explicitly models repeat recovery.
6. **Don't waste origin cargo.** Origin is only deducted if at least one delivery item can be accepted at the destination. If destination is completely full, the cycle is skipped and origin pays nothing. Per-item delivery is independent: each item fills what fits, and shortfalls are logged and shown instead of silently disappearing.
7. **Dock + deliver + undock required in v1.** A route can only be created from a recording chain where the transport forms a detected docking connection to an endpoint, delivery cargo moves from the transport into the endpoint while docked, the endpoint retains that cargo through undock, and the transport undocks afterward. Claw/grapple and stock crossfeed/fuel-line paths are deferred until docking routes are reliable.

2.3 Abstraction model

8. **No physical vessel during transit.** Route execution is pure math — deduct at origin, wait, add at destination. The ghost is visual only. No physics, no collisions, no orbit propagation.
9. **Endpoints are vessels first, locations second.** Endpoint PID is the primary identity. Surface endpoints may fall back to a single nearest compatible vessel near the recorded coordinates, but Parsek does not create an abstract area warehouse. Orbital endpoints use PID only. This keeps delivery close to stock vessel semantics while still tolerating surface base rebuilds.
10. **The system doesn't produce cargo.** Parsek moves resources and inventory items between stock vessels. Mining, ISRU, solar power, manufacturing, and crew hiring are the player's responsibility. Routes chain naturally: the output of one feeds the input of another.

2.4 Timeline integration

11. **Dispatches and deliveries are timeline events.** Route activity participates in the same ledger and epoch isolation system as game actions. Reverts invalidate dispatches from abandoned timelines. Stock save

rollback handles stock loads; Parsek timeline rewinds recompute funds/resources through the effective ledger.

12. **Routes persist across scenes and save/load.** All route state is serialized in the .sfs. The scheduler runs in all scenes via ParsekScenario.

3. Terminology

Logistics — the player-facing feature/window that manages Supply Routes.

Supply Route — a separate entity that defines one recurring cargo transfer from an origin to one endpoint. Created from a committed Supply Run after player confirmation. Uses chain-sequential ghost playback (not the per-recording loop system).

Supply Run — the concrete player-flown recording chain that proves a route. It contains the transport path, stock connection window, resource/inventory delta, and disconnect. In v1 the connection window is dock/undock.

Route stop / endpoint — the destination vessel and location for the route. v1 exposes one endpoint per Supply Route. The data model keeps a `Stops` list so multi-stop Supply Runs can be added later without replacing the save shape.

Stock connection window — the bounded time interval where the transport is connected to the endpoint by a stock mechanism and cargo can move. Docking port dock/undock is the only v1 window. Claw/grapple and other stock transfer paths are future producers for the same interface once detection is proven.

Endpoint — origin or destination location of a route. Defined by body, coordinates, and the target vessel PID from the stock connection window. Surface endpoints can fall back to one nearest compatible vessel near the recorded coordinates. Orbital endpoints use PID only.

Delivery manifest — the per-resource and inventory amounts delivered at the endpoint. v1 is delivery-only: resource and inventory deltas must represent cargo leaving the transport and appearing on the endpoint part set while docked. Pickup routes and mixed pickup/delivery windows are deferred.

Cost manifest — the resource and inventory quantities used or delivered by the Supply Run. For non-KSC origins, this is deducted from the recorded start-docked origin depot each cycle.

KSC dispatch cost — in Career, the funds cost charged when a KSC-origin route dispatches. It is computed from stock costs for the source vessel parts plus the resource/inventory quantities used or delivered by the Supply Run.

Dispatch — the moment a route cycle begins. Origin resources are checked and deducted. A timeline event is created.

Delivery — the moment delivery occurs at a stop (between chain segments). Resources and inventory are added to the endpoint vessel. A timeline event is created.

Round-trip link — future scheduling constraint pairing two one-way routes: "don't dispatch me until my partner completes."

Route analysis engine — pure logic that walks the committed Supply Run chain to extract the endpoint, delivery manifest, source recording IDs, and transit duration from recording data (resource/inventory manifests, stock connection target PID, connection kind, location context).

4. Data Model

4.1 ResourceAmount

```
internal struct ResourceAmount
{
    public double amount;
    public double maxAmount;
}
```

4.2 InventoryPayloadItem

```
internal class InventoryPayloadItem
{
    public string PartName;           // stock part name from STOREDPART
    public string VariantName;        // variant/theme identity when present
    public int Quantity;              // stock quantity for identical payload
    snapshots
    public int SlotsTaken;             // stock inventory slot cost
    public double UnitDryCost;         // dry part/module cost, excluding stored
    resources
    public Dictionary<string, double> StoredResources; // resources carried inside the
    stored part
    public string IdentityHash;        // hash of the canonical STOREDPART snapshot
    public ConfigNode StoredPartSnapshot; // full stock STOREDPART node for reconstruction
}
```

Inventory route manifests use exact stored-part payload snapshots, not part-name counts. Two stored parts with different variants, resource contents, module state, or stock snapshot data are different payload items. `Quantity` may compress identical snapshots only after the canonical `STOREDPART` payload is equal.

4.3 RouteEndpoint

```
internal struct RouteEndpoint
{
    public string bodyName;           // e.g., "Kerbin", "Mun"
    public double latitude;           // from dock event trajectory point
    public double longitude;
    public double altitude;
    public uint vesselPid;            // 0 = KSC sentinel for KSC origins; non-zero for all
    stops and non-KSC origins
    public bool isSurface;            // true = landed/splashed/prelaunch endpoint; enables
    surface fallback
}
```

`vesselPid = 0` is reserved for KSC origins where there is no stock origin vessel to resolve. Non-KSC origins and all route stops must use a non-zero vessel PID captured from the stock connection boundary.

4.4 RouteConnectionKind

```
internal enum RouteConnectionKind
{
    DockingPort,
    Grapple,          // stock Advanced Grabbing Unit / claw, if target detection is reliable
    StockCrossfeed, // stock transfer/crossfeed path, if endpoint detection is reliable
    Unknown
}
```

4.5 RouteStatus

```
internal enum RouteStatus
{
    Active,           // dispatching on schedule
    InTransit,        // dispatched, waiting for transit duration to elapse
    WaitingForResources, // origin exists but lacks resources – delayed
    WaitingForFunds,   // Career KSC-origin route lacks dispatch funds – delayed
    DestinationFull,   // destination can't accept delivery – skipping cycles
    EndpointLost,      // destination/origin vessel gone (orbital PID miss or no surface
                      // vessels)
    MissingSourceRecording, // route source recording chain is gone; route cannot dispatch
    Paused              // player manually paused
}
```

4.6 RouteStop

```
internal class RouteStop
{
    public RouteEndpoint Endpoint;           // where this stop is
    public RouteConnectionKind ConnectionKind; // how the Supply Run connected
    public Dictionary<string, double> DeliveryManifest; // per-resource delivery amounts
    (positive only in v1)
    public List<InventoryPayloadItem> InventoryDeliveryManifest; // exact stored-part
    payloads delivered in v1
    public int SegmentIndexBefore;           // 0-based source recording
    whose completion UT triggers this stop
}
```

v1 exposes a single-stop route in the UI. Multi-stop Supply Runs are a planned extension; the list shape stays now so save data does not need to be replaced later.

4.7 Route

```
internal class Route
{
    // Identity
    public string Id;           // unique route ID (GUID)
    public List<string> RecordingIds; // ordered chain of source recording IDs
    public string Name;         // player-visible name (editable)
```

```

// Endpoints
public RouteEndpoint Origin;
public List<RouteStop> Stops;           // ordered stops along the route
public bool IsKscOrigin;                // true = Career charges KSC funds instead of
physical origin cargo

// Resource transfer
public Dictionary<string, double> CostManifest; // per-resource quantities used or
delivered
public List<InventoryPayloadItem> InventoryCostManifest; // exact stored-part payloads
used or delivered
public double KscDispatchFundsCost;        // stock part + used/delivered
cargo funds per KSC dispatch

// Timing
public double TransitDuration;            // seconds (= total chain duration)
public double DispatchInterval;          // seconds between cycle starts
public double DispatchWindowEpochUT;    // original flight start UT; anchors inter-body
synodic phase
public double DispatchWindowPeriod;      // 0 for same-body, synodic period for inter-body
public double NextDispatchUT;            // UT of next scheduled dispatch
public double? CurrentCycleStartUT;      // UT when the in-transit cycle began; null when
idle
public double? NextEligibilityCheckUT;    // retry backoff for resource/funds waits; null
when not waiting
public int CurrentSegmentIndex;          // 0-based active source-recording index; -1 when
not in transit

// Per-stop pending delivery (computed at each stop boundary during transit)
public double? PendingDeliveryUT;        // UT when next route boundary is due (null if not
in transit)
public int PendingStopIndex;             // stop due at PendingDeliveryUT, or -1 when
current boundary has no stop

// Linking
public string LinkedRouteId;             // paired route for round-trip (null if standalone)

// State
public RouteStatus Status;
public bool PauseAfterCurrentCycle;      // pause requested while InTransit; transition to
Paused after completion
public int CompletedCycles;              // total successful cycle completions
public int SkippedCycles;                // cycles skipped (destination full, origin empty)
}

```

Forward compatibility with multi-stop routes: A v1 Supply Route has `Stops.Count == 1`. Later multi-stop routes can reuse the same list without changing the top-level route save node.

Route playback segment definition: In this document, a route playback segment is one source recording in `RecordingIds`. `CurrentSegmentIndex` indexes `RecordingIds` directly: `-1` means no active transit, `0` means the first source recording is active, `1` means the second source recording is active, and so on. It does not index

trajectory samples, internal `TrackSection` ranges, or lower-level chain-manager segments.

`RouteStop.SegmentIndexBefore` is the source-recording index whose completion boundary triggers that stop.

4.8 Serialization format

```
ROUTE
{
  id = <guid>
  name = Mun Fuel Run

  RECORDING_IDS
  {
    id = <recording-guid-1>
    id = <recording-guid-2>
    id = <recording-guid-3>
  }

  ORIGIN
  {
    bodyName = Kerbin
    latitude = -0.0972
    longitude = -74.5577
    altitude = 75.2
    vesselPid = 0
    isSurface = True
  }

  STOP
  {
    ENDPOINT
    {
      bodyName = Mun
      latitude = 3.2001
      longitude = -45.1234
      altitude = 612.5
      vesselPid = 67890
      isSurface = True
    }
    connectionKind = DockingPort
    segmentIndexBefore = 1
    DELIVERY_MANIFEST
    {
      LiquidFuel = 150.0
      Oxidizer = 183.3
    }
    INVENTORY_DELIVERY_MANIFEST
    {
      ITEM
      {
        partName = smallSolarPanel
        variantName =
```

```

        quantity = 2
        slotsTaken = 2
        unitDryCost = 75.0
        identityHash = 8F17...
        STOREDPART
        {
            // canonical stock STOREDPART snapshot, including variant/module/resource
payload
        }
    }
}
}
isKscOrigin = True
kscDispatchFundsCost = 12500.0
transitDuration = 12345.6
dispatchInterval = 43200.0
dispatchWindowEpochUT = 42654.0
dispatchWindowPeriod = 0.0
nextDispatchUT = 55000.0
// currentCycleStartUT omitted when null
// nextEligibilityCheckUT omitted when null
currentSegmentIndex = -1
// pendingDeliveryUT omitted when null
pendingStopIndex = -1
linkedRouteId =
status = Active
pauseAfterCurrentCycle = False
completedCycles = 5
skippedCycles = 1

COST_MANIFEST
{
    LiquidFuel = 155.0
    Oxidizer = 183.3
}
INVENTORY_COST_MANIFEST
{
    ITEM
    {
        partName = smallSolarPanel
        variantName =
        quantity = 2
        slotsTaken = 2
        unitDryCost = 75.0
        identityHash = 8F17...
        STOREDPART
        {
            // same canonical payload identity used for delivery and cost
        }
    }
}
// KSC-origin routes use ORIGIN.vesselPid = 0. Non-KSC origins and all STOP endpoints use

```

```

real vessel PIDs.
    // When Status == InTransit, pendingDeliveryUT/pendingStopIndex identify the next due
    boundary.
    // When Status == InTransit, currentCycleStartUT identifies the scheduler elapsed time for
    visual handoff.
    // When Status == WaitingForResources or WaitingForFunds, nextEligibilityCheckUT gates
    retry polling.
    // Actual deliverable amounts are recomputed at delivery time from current endpoint
    capacity.
}

```

Routes are stored in their own `ROUTES` `ConfigNode` section inside `ParsekScenario`'s save data, alongside recordings and game actions. Additive — saves without routes load fine. Routes reference recordings by ID but are independent entities. A route whose source recordings are missing is disabled, not allowed to keep moving cargo without the proof recording.

4.9 Recording extensions (Phase 11 + Logistics prerequisites)

Phase 11 adds base manifest metadata to recordings. Manifests are captured at segment boundaries (recording start, end, dock/undock events) from the vessel snapshot that already exists at those points. Logistics also needs connection-scoped manifests: during docking, KSP merges vessels, so aggregate vessel resources cannot prove that cargo moved between the transport and endpoint. Route analysis must filter manifests by the original transport part persistent IDs and original endpoint part persistent IDs captured at the docking boundary, then resolve those same part sets after undock.

Resource manifests (v1 — Phase 11):

```

public Dictionary<string, ResourceAmount> StartResources; // manifest at recording start
public Dictionary<string, ResourceAmount> EndResources;   // manifest at recording end

```

`ResourceAmount` is a struct with `amount` and `maxAmount` fields. Resources are summed across all parts. `ElectricCharge` and `IntakeAir` are excluded (environmental noise). Extracted by `VesselSpawner.ExtractResourceManifest(ConfigNode vesselSnapshot)`.

Connection-scoped manifests (v1 logistics prerequisite):

```

public List<uint> TransportPartPersistentIds; // original transport part set
public List<uint> EndpointPartPersistentIds; // original endpoint part set
public Dictionary<string, ResourceAmount> DockTransportResources;
public Dictionary<string, ResourceAmount> UndockTransportResources;
public Dictionary<string, ResourceAmount> DockEndpointResources;
public Dictionary<string, ResourceAmount> UndockEndpointResources;
public List<InventoryPayloadItem> DockTransportInventory;
public List<InventoryPayloadItem> UndockTransportInventory;
public List<InventoryPayloadItem> DockEndpointInventory;
public List<InventoryPayloadItem> UndockEndpointInventory;
public Vessel.Situations TransferEndpointSituation; // endpoint vessel situation
at docking
public uint StartDockedOriginVesselPid; // non-KSC origin depot, if
recording starts docked

```

The docked aggregate vessel manifest is still useful for diagnostics, but Logistics v1 must compute delivery from matched transport-loss and endpoint-gain fields. Cost uses full-run fields scoped to the original transport part set. Missing connection-scoped fields mean the recording cannot become a Supply Route.

Serialized as:

```
RESOURCE_MANIFEST
{
  RESOURCE { name = LiquidFuel  startAmount = 3600  startMax = 3600  endAmount = 200  endMax
= 3600 }
  RESOURCE { name = Oxidizer    startAmount = 4400  startMax = 4400  endAmount = 244  endMax
= 4400 }
}
```

Stock connection target vessel PID (v1 logistics prerequisite):

```
public uint TransferTargetVesselPid; // PID of vessel connected to at this segment
boundary (0 = no route-relevant connection)
public RouteConnectionKind TransferKind; // DockingPort only in v1; future-proofed for
later producers
```

For the first implementation, `TransferTargetVesselPid` can be populated from the existing docking-path capture that currently behaves like `DockTargetVesselPid`. The route-facing field name should be generic because the route concept is not inherently dock-only, but v1 should reject any `TransferKind` other than `DockingPort`. Claw/grapple and stock crossfeed/fuel-line support can add producers for the same connection-target contract later instead of creating separate route types.

Inventory manifests (implemented — Phase 11, extended for Logistics):

KSP 1.12 `ModuleInventoryPart` items (stored parts in cargo containers). The base Phase 11 manifest can count part names and slot usage for diagnostics, but Logistics v1 requires exact payload snapshots.

`ExtractInventoryManifest` walks `MODULE > STOREDPARTS > STOREDPART` nodes, preserves each canonical `STOREDPART ConfigNode`, records slot usage, variant identity, stored resources, and a stable identity hash. Delivery and cost matching operate on those payload identities, not just part names. This enables automated parts delivery without treating variants, stacked payloads, or resource-filled stored parts as fungible.

Crew manifests (implemented — Phase 11, not consumed by Logistics v1):

Crew composition by trait (Pilot/Scientist/Engineer/Tourist). Logistics v1 does not deliver generic kerbals; stock KSP crew are named roster entities, not fungible cargo. Crew rotation should be a later feature with explicit roster, hiring, and reservation semantics.

All manifest types are additive — missing node = no data. No format version bump.

5. Route Creation

5.1 Supply Run workflow

Route creation uses an automatic post-commit prompt on an eligible Supply Run. A special pre-flight mode is not required for v1.

Player flow:

1. Player flies mission normally.
2. Transport docks to the destination.
3. Player transfers resources and/or stock inventory to the destination through KSP's normal UI/systems.
4. Transport undocks from the destination so the endpoint is available for the next dispatch.
5. Player commits the recording.
6. If route analysis finds an eligible Supply Run, Parsek automatically prompts "Create Supply Route from this Supply Run?"
7. Player reviews origin, endpoint, delivery manifest, KSC dispatch cost, transit time, and interval; then confirms.
8. Supply Route goes live.

Deferred "Record Supply Run" helper:

- May mark the current recording tree as route-intended for UI filtering and prompt suppression.
- Shows a "Supply Run Active" indicator.
- Does not change recorder behavior. The same `FlightRecorder`, chain boundaries, snapshots, and manifests are used.
- Does not make an invalid run valid. The committed chain must still contain a detected docking connection, delivery cargo delta, and undock.

5.2 Route analysis engine

`RouteAnalysisEngine` (in `Logistics/`) is pure logic over committed Recording fields. Fully testable without KSP.

Input: ordered list of recordings in the committed Supply Run chain. This can come from an explicit route-intended session marker, the just-committed tree, or a "Create Supply Route" action on an existing recording tree.

Algorithm:

Walk chronologically through the chain:

For each complete DockingPort connection window:

Find the matching undock boundary after the docking boundary

`stop.endpoint = {`

body, lat, lon from connection-time trajectory point,

`vesselPid = TransferTargetVesselPid,`

`connectionKind = TransferKind,`

`isSurface = connected endpoint vessel situation is Landed/Splashed/Prelaunch`

`}`

`transportBefore = resource/inventory manifest for the original transport part PID set`
captured immediately before docking merge

`transportAfter = resource/inventory manifest for the same transport part PID set`
captured immediately after undock/separation

`endpointBefore = resource/inventory manifest for the original endpoint part PID set`
captured immediately before docking merge

`endpointAfter = resource/inventory manifest for the same endpoint part PID set`
captured immediately after undock/separation

`transportLoss = positive deltas from transportBefore - transportAfter`

`transportGain = positive deltas from transportAfter - transportBefore`

`endpointGain = positive deltas from endpointAfter - endpointBefore`

`endpointLoss = positive deltas from endpointBefore - endpointAfter`

if any non-ignored transportGain or endpointLoss exists:

reject candidate as pickup/mixed-transfer window in v1

`stop.deliveryManifest = per-resource min(transportLoss, endpointGain)`

(delivery proof is connection-scoped, not merged-vessel aggregate)

`stop.inventoryDeliveryManifest = exact InventoryPayloadItem snapshots matched between`

```

                                transport loss and endpoint gain by identityHash
stop.transitDuration = delivery boundary UT - recording start UT

origin = KSC launch site, or non-KSC origin depot proven by a start-docked origin vessel
totalTransit = last recording EndUT - first recording StartUT
costManifest = source resource quantities used or delivered over the full run
inventoryCostManifest = exact inventory payload snapshots used or delivered over the full
run

```

Derived values:

- **Origin** = KSC launch site, or a non-KSC origin depot vessel if the Supply Run starts docked to that depot and records its PID
- **Stops** = the one complete dock-transfer-undock delivery window accepted by v1; multiple delivery windows make the candidate ineligible
- **Origin vessel PID** = 0 for KSC routes; otherwise the start-docked origin depot vessel PID
- **Stop vessel PID** = TransferTargetVesselPid from the stock connection boundary
- **IsKscOrigin** = true if origin body is Kerbin and coordinates are near a launch site
- **DeliveryManifest** = matched resource amount that both left the transport part set and appeared on the endpoint part set across the dock/undock window
- **InventoryDeliveryManifest** = exact stored-part payload snapshots that both left the transport part set and appeared on the endpoint part set across the same window
- **CostManifest** = source resource quantities used or delivered over the full Supply Run
- **InventoryCostManifest** = exact stored-part payload snapshots used or delivered over the full Supply Run
- **KscDispatchFundsCost** = dry stock part cost plus stock resource/inventory cost for used-or-delivered quantities on Career KSC-origin routes
- **TransitDuration** = total chain duration (last recording EndUT minus first recording StartUT)
- **DispatchInterval** = TransitDuration (default; player can increase). For inter-body routes: defaults to synodic period of origin and destination bodies.
- **DispatchWindowEpochUT** = first recording StartUT; inter-body repeats stay phase-aligned to this UT
- **DispatchWindowPeriod** = 0 for same-body routes, synodic period for inter-body routes
- **RecordingIds** = ordered list of all recording IDs in the chain

Cost calculation:

For v1, `CostManifest` is the positive decrease in the source transport's non-ignored resources over the full Supply Run, computed over the original transport part PID set rather than the aggregate docked vessel:

```

CostManifest[resource] = max(0, startTransportResources[resource] -
endTransportResources[resource])

```

This includes delivered cargo plus transit resources consumed by the flown mission. `InventoryCostManifest` uses the same principle for exact stock stored-part payload snapshots that leave the transport by the end of the run.

For KSC-origin Career routes, `KscDispatchFundsCost` is:

```

dry source transport part cost
+ stock resource unitCost * CostManifest amount
+ dry stock part cost for InventoryCostManifest items

```

Dry part cost means the source transport's parts with stored resources emptied and inventory contents excluded, so resource and inventory values are not double-counted. Inventory-contained parts use `InventoryPayloadItem.UnitDryCost` plus `StoredResources` from their captured `STOREDPART` snapshot, so a

solar panel variant, a stacked spare wheel, and a resource-filled stored part are costed from the same payload identity that delivery will reconstruct. v1 does not apply recurring recovery credit.

Routes store `CostManifest` and `KscDispatchFundsCost` together for transparency and future re-costing. On KSC-origin routes, `CostManifest` is not physically deducted; `KscDispatchFundsCost` is derived from that same source data. If serialized values disagree after load/revalidation, treat that as a bug and recompute from the source recording rather than honoring the desync.

5.3 Validation

The route analysis pass validates the chain:

1. At least one docking connection boundary exists (`TransferTargetVesselPid != 0` and `TransferKind == DockingPort`)
2. A matching undock boundary exists after the docking boundary
3. At least one resource or exact stock stored-part payload both left the original transport part PID set and appeared on the original endpoint part PID set between the dock and undock snapshots
4. The source recording chain is present and playable
5. v1: exactly one delivery window is present. If two or more dock-transfer-undock delivery windows are detected, validation rejects the candidate and reports that multi-stop Supply Routes are deferred
6. v1: no pickup deltas are present in the same connection window. Formally, after EC/IntakeAir filtering, any positive `transportGain` or `endpointLoss` rejects the candidate as pickup or mixed transfer
7. v1: non-KSC origins are eligible only when the recording starts docked to a real origin depot vessel and captures that vessel PID; non-KSC candidates without that proof are rejected

If validation fails, the route confirmation UI shows what's missing (e.g., "Transport must undock from destination to enable route — the endpoint needs to be free for the next cycle").

5.4 Player confirmation

Route configuration panel shows derived values: origin, endpoint, delivery manifest, total transit time, origin cost manifest, KSC dispatch funds cost, and connection kind. Player can edit name, dispatch interval, and enable/disable. On confirm, route is created and scheduling begins.

5.5 Multi-stop routes

Multi-stop routes are a natural extension but not required for v1. The analysis engine can detect multiple dock-transfer-undock delivery windows and report them in diagnostics, but v1 must reject the route candidate instead of exposing only the first stop. This avoids charging/deducting the full Supply Run cost while delivering only part of the run, and keeps partial-failure semantics out of v1.

Multi-stop example:

```
Player flies: KSC -> Base A (dock, deliver 150 LF) -> Base B (dock, deliver spare parts) -> KSC
```

Future route analysis produces:

```
Origin: KSC, Kerbin
Stop 1: Base A on Mun -- delivers 150 LF, 183 Ox
Stop 2: Base B on Mun -- delivers spare parts
Total transit: 2d 4h
Round-trip: Yes
```

6. Dispatch and Delivery

6.1 Dispatch evaluation

Trigger: The route scheduler (`RouteScheduler`) runs each physics frame (or once per second during warp) in all scenes via `RouteOrchestrator` , called from `ParsekScenario.Update` .

For each route with Status in {Active, WaitingForResources, WaitingForFunds, DestinationFull} and `NextDispatchUT <= currentUT` :

Routes with Status `InTransit`, `Paused`, `EndpointLost`, or `MissingSourceRecording` are excluded from dispatch evaluation.

For `WaitingForResources` and `WaitingForFunds` , skip evaluation until `NextEligibilityCheckUT == null || NextEligibilityCheckUT <= currentUT` . When a wait condition is found, set `NextEligibilityCheckUT = currentUT + 60s` by default. Save/load preserves the value. Route UI actions, resource-store changes, funds ledger changes, and route revalidation may clear it to force an immediate retry.

`MissingSourceRecording` routes are not retried as normal dispatch candidates. They are revalidated on save load, when the recordings store changes, and from a route UI "Revalidate" action. If every source recording ID resolves again, the route returns to `Active` and `NextDispatchUT` is recalculated from current UT.

Step 1: Ignore reserved round-trip fields in v1. `LinkedRouteId` is serialized for forward compatibility only. v1 dispatch does not check it.

Step 2: Check source recordings. Verify every `RecordingIds` entry still resolves to a committed recording. If not, set `Status = MissingSourceRecording` and stop. No proof recording means no cargo transfer.

Step 3: Check destination. Find the endpoint vessel (section 7). If NO vessel is found -> set `Status = EndpointLost` , skip cycle. If the vessel has zero capacity for all delivery resources and inventory items -> set `Status = DestinationFull` , increment `SkippedCycles` , advance `NextDispatchUT` . If capacity is available -> proceed.

Step 4: Check origin. For non-KSC origins, find the start-docked origin depot vessel (section 7). If no vessel is found -> set `Status = EndpointLost` , skip cycle. If the vessel lacks `CostManifest` resources or `InventoryCostManifest` items -> set `Status = WaitingForResources` , set `NextEligibilityCheckUT` , and do NOT advance `NextDispatchUT` . For KSC origins in Career, check that `KscDispatchFundsCost` is affordable under the existing ledger reservation rules. If funds are insufficient, set `Status = WaitingForFunds` , set `NextEligibilityCheckUT` , and do NOT advance `NextDispatchUT` . For KSC origins in Science or Sandbox, skip the funds branch entirely; do not touch `Funding.Instance` .

Step 5: Dispatch. Clear `NextEligibilityCheckUT` . Deduct `CostManifest` / `InventoryCostManifest` from non-KSC origin, or charge `KscDispatchFundsCost` for KSC origin in Career. Science and Sandbox KSC origins pay no funds. Set `CurrentCycleStartUT` to the scheduled dispatch UT being processed (`NextDispatchUT`), set `CurrentSegmentIndex = 0` , compute `PendingDeliveryUT` from the first route boundary, and set `PendingStopIndex` to the stop due at that boundary or `-1` . Tell the ghost playback engine to play the first recording in `RecordingIds` when visuals are available. Set `Status = InTransit` . Create `ROUTE_DISPATCHED` timeline event at `CurrentCycleStartUT` . Advance `NextDispatchUT` .

6.2 UT-driven chain progression

The route scheduler, not ghost playback, is authoritative for route state. `RouteOrchestrator.Tick(currentUT)` processes in-transit routes in all scenes and during time warp:

1. If any source recording is missing, set `Status = MissingSourceRecording` , stop playback if active, and abort without delivery.
2. While `Status == InTransit` and `PendingDeliveryUT <= currentUT` , process the due boundary.
3. If `PendingStopIndex >= 0` , execute delivery for that stop using current endpoint capacity.
4. Advance `CurrentSegmentIndex` to the next 0-based source recording index and compute the next `PendingDeliveryUT / PendingStopIndex` .
5. If the last source recording boundary was processed, increment `CompletedCycles` , reset `CurrentCycleStartUT = null` , `CurrentSegmentIndex = -1` , `PendingDeliveryUT = null` , and `PendingStopIndex = -1` . If `PauseAfterCurrentCycle` is true, clear it and set `Status = Paused` ; otherwise set `Status = Active` .

`OnPlaybackCompleted` remains a visual integration hook only. It can let the route scheduler start the next ghost source recording promptly in flight, but it must not be the only path that advances route state. Save/load and high time warp are handled by the UT-driven tick loop.

Routes do NOT use the per-recording loop toggle. The route scheduler owns all timing and sequencing. Routes and loops are siblings — both use the ghost playback engine, but routes chain multiple trajectories sequentially with delivery logic between segments.

Visual handoff when entering flight mid-transit: The scheduler elapsed time is `currentUT - CurrentCycleStartUT` . When visuals become available for an already in-transit route, the route policy resolves that elapsed time to a `RecordingIds` index and a recording-local offset, then starts/seekes ghost playback at that offset. It must not restart the ghost at recording UT 0 after the route has already advanced. If the current cycle has no remaining visible source recording to show, the route simply skips ghost rendering until the next source recording or cycle; delivery state still advances by UT.

6.3 Per-stop delivery execution

Trigger: `RouteOrchestrator.Tick(currentUT)` reaches a pending stop boundary. In flight, the same moment normally coincides with ghost source-recording completion; outside flight, delivery still happens by UT.

1. **Re-check source recordings.** If any `RecordingIds` entry is missing, set `Status = MissingSourceRecording` , create no delivery event, and abort. No proof recording means no cargo transfer, even mid-transit.
2. **Find endpoint vessel** at the route endpoint (section 7). If no vessel is found after dispatch, log a warning and create a `ROUTE_DELIVERY_FAILED` timeline event. Transit cost has already been paid; no cargo is conjured.
3. **For each resource in the stop's `DeliveryManifest`** : apply to the endpoint vessel tanks, clamped to current `maxAmount` . v1 manifests contain positive delivery amounts only. For unloaded vessels: modify `ProtoPartResourceSnapshot.amount` directly, respect `flowState` . For loaded vessels: use `Part.RequestResource()` .
4. **Deliver inventory** by reconstructing exact `InventoryPayloadItem.StoredPartSnapshot` payloads into stock `ModuleInventoryPart` slots. Items that do not fit remain undelivered and are reported in the route event/log.
5. **Create `ROUTE_DELIVERED` timeline event.** Record requested and actual amounts so the player can see partial fills instead of silent loss.

6.4 Single-delivery execution

For v1, the single-stop route is the only player-facing shape. Delivery executes once when scheduler UT reaches the recorded boundary after `SegmentIndexBefore` .

6.5 Per-resource independent delivery

Each delivery resource at each stop is evaluated independently:

```
For each resource in stop.DeliveryManifest:
    capacity = sum of (maxAmount - amount) across all stop vessel tanks for this resource
    deliver = min(manifest amount, capacity)
    actualDelivery[resource] = deliver
```

Origin cost is paid at dispatch time after destination capacity has been checked. Non-KSC origins deduct `CostManifest / InventoryCostManifest` from the proven origin depot. KSC-origin Career routes charge `KscDispatchFundsCost` instead of deducting physical cargo from KSC. Science and Sandbox KSC origins dispatch with no funds charge.

Inventory follows the same independent rule at item granularity: deliver what fits, report what did not fit, never create extra slots or abstract storage.

6.6 Pause, unpause, and re-target

Pause: Player clicks Pause in route UI -> `Status = Paused` for future dispatches. If a dispatch is already `InTransit`, set `PauseAfterCurrentCycle = true` and keep `Status = InTransit` until the cycle finishes. Delivery still executes at the endpoint; after the final boundary is processed, the route transitions to `Paused` instead of `Active`. This matches stock expectations: the supply vessel has already launched / departed, so pausing the route should not freeze cargo in mid-flight.

Unpause: Player clicks Resume -> `Status = Active`. If the route is still `InTransit` with `PauseAfterCurrentCycle = true`, Resume clears that flag and the in-flight cycle will finish back to `Active`. Route re-enters dispatch evaluation on next scheduler tick. `NextDispatchUT` is recalculated if stale (advanced to next valid dispatch time from currentUT).

Cancel current dispatch: Deferred. If added later, cancellation should be explicit and should not refund already-deducted origin cargo unless the route event model explicitly records a reversible failure.

Re-target (EndpointLost recovery): Player selects a new destination vessel in the route UI -> endpoint coordinates and vesselPid updated, `Status` transitions from `EndpointLost` to `Active`. Same mechanism for origin re-targeting on non-KSC routes. Re-targeting is an explicit player intent declaration, not automatic proof that the original Supply Run visited the new vessel. The route keeps its recorded delivery and cost manifests, logs the re-target, and the UI should warn that future cycles reuse the proven transport/cargo run against the newly selected endpoint or origin. Automated surface fallback remains deliberately tight; broad endpoint generalization requires this explicit player action.

7. Endpoint Resolution

7.1 Algorithm

`RouteEndpointResolver.SurfaceFallbackRadiusMeters = 50.0` in v1. The radius is deliberately tight: it allows rebuilding a surface base in-place, but avoids treating neighboring pads, rovers, or storage craft as one abstract warehouse. If this proves too restrictive for large surface installations, make it a settings-backed value later rather than silently widening v1 matching.

KSC-origin routes do not resolve an origin vessel: `IsKscOrigin == true` and `Origin.vesselPid == 0` mean "charge/skip KSC funds branch according to game mode." A route stop endpoint or non-KSC origin with `vesselPid == 0` is invalid.

```
ResolveEndpointVessel(endpoint):
```

1. Find vessel by `endpoint.vesselPid` in `FlightGlobals.Vessels`
→ if found and compatible: return vessel
2. If not found AND `endpoint.isSurface == false`:
→ return null (orbital endpoints have no fallback)
3. If not found AND `endpoint.isSurface`:
→ scan `FlightGlobals.Vessels` for the nearest compatible vessel within `RouteEndpointResolver.SurfaceFallbackRadiusMeters` of (body, lat, lon, alt)
→ return that vessel, or null

7.2 Surface vs orbital behavior

- **Surface endpoints:** PID primary, single nearest compatible-vessel fallback within `SurfaceFallbackRadiusMeters`. `isSurface` is captured from the endpoint vessel situation (`Landed` , `Splashed` , or `Prelaunch`), not altitude, so Mun/Minmus surface bases remain surface endpoints. Handles base rebuilding without turning every nearby vessel into one abstract warehouse.
- **Orbital endpoints:** PID only. Orbital coordinates change every second, so proximity fallback does not work. If an orbital station is destroyed and rebuilt, the player must re-target the route.

7.3 Compatible vessel definition

A compatible endpoint or origin fallback vessel must be a real stock vessel, not a Parsek ghost/map-presence vessel, and must be a vessel type that can plausibly own cargo. Exclude vessels in `GhostMapPresence` tracking sets and stock `VesselType` values such as `EVA` , `Flag` , `Debris` , `SpaceObject` , and `Unknown` . For destination resolution, the vessel must contain at least one eligible tank for a delivered resource or one `ModuleInventoryPart` that can store a delivered payload type, regardless of current free capacity; full-but-compatible destinations are handled by the destination-capacity check, not treated as lost endpoints. For non-KSC origin resolution, the vessel must contain at least one eligible store for the route cost manifest. If several compatible vessels remain within the surface fallback radius, choose the nearest one only.

7.4 Loaded vs unloaded vessels

If the endpoint vessel is loaded (player is within physics range), use `Part.RequestResource()` for resource operations. If unloaded, use `ProtoPartResourceSnapshot.amount` directly. Both paths apply to origin (deduction) and destination (delivery).

Inventory delivery also has loaded and unloaded paths. For loaded vessels, use stock `ModuleInventoryPart` APIs to add/remove reconstructed `STOREDPART` payloads while respecting slot limits. For unloaded vessels, edit the relevant `ProtoPartModuleSnapshot` / `STOREDPARTS` `ConfigNodes` directly using the stored `InventoryPayloadItem.StoredPartSnapshot` , then update slot accounting. Both paths preserve the exact payload identity hash and report items that do not fit.

8. Transfer Window Scheduling

8.1 Synodic period computation

```
SynodicPeriod(originBody, destBody):  
    if originBody == destBody:  
        return 0 // same body, no transfer window  
    if originBody is Sun or destBody is Sun:
```

```

    return 0 // no stable parent orbit to compare
// Walk up to common parent
a = originBody, b = destBody
while hierarchy depth of a > hierarchy depth of b: a = a.referenceBody
while hierarchy depth of b > hierarchy depth of a: b = b.referenceBody
while a.referenceBody != b.referenceBody:
    a = a.referenceBody
    b = b.referenceBody
// a and b now orbit the same parent
T1 = a.orbit.period, T2 = b.orbit.period
if T1 == T2: return 0
return abs(1 / (1/T1 - 1/T2))

```

Handles cross-system routes: Mun→Laythe walks up to Kerbin/Jool orbiting Sun, then uses the Kerbin/Jool synodic period. Guards routes directly to/from the Sun and equal-period edge cases.

8.2 Dispatch interval rules

- **Same-body routes:** Player-set interval. Minimum = recording duration.
- **Inter-body routes:** Default = synodic period, phase-anchored to the original Supply Run start UT (`DispatchWindowEpochUT`). `NextDispatchUT` is always the smallest `DispatchWindowEpochUT + n * DispatchWindowPeriod` that is $\geq$ current UT and also respects the player's minimum interval.
- **Player interval override:** Player can increase the minimum spacing between dispatches, but v1 does not shift the transfer-window phase. A later advanced override may allow phase shifting explicitly.
- **Gravity assist routes:** Two-body synodic approximation (intermediate flybys not tracked). Player can fine-tune by increasing minimum spacing, not by changing the phase anchor in v1.

9. Round-Trip Linking (Future)

Round-trip linking is not v1 implementation scope. In v1, `LinkedRouteId` is a reserved serialization field only: route creation does not expose linking UI, dispatch ignores `LinkedRouteId`, and tests only verify that the field round-trips without changing behavior.

Future design intent:

- A round trip remains two separate one-way Supply Routes, each created from its own Supply Run.
- A later UI may link two routes as a pair so they alternate dispatch eligibility: Route A completes -> Route B may dispatch -> Route B completes -> Route A may dispatch.
- Future dispatch rules may wait while the linked partner is `InTransit`. Pausing a partner should not block the other unless that future design explicitly changes the rule.

10. Edge Cases

10.1 Destination destroyed, surface base

Scenario: Mun base destroyed. Player rebuilds at same spot. **Behavior:** PID match fails. Surface proximity fallback finds new vessel within `SurfaceFallbackRadiusMeters`. Route auto-reconnects. **v1 limitation:** If rebuilt outside `SurfaceFallbackRadiusMeters`, player must explicitly re-target or create a new route.

10.2 Destination destroyed, orbital station

Scenario: Kerbin station destroyed. Player rebuilds. **Behavior:** PID match fails. No proximity fallback. `Status = EndpointLost` . Player must re-target.

10.3 Origin destroyed or recovered (non-KSC)

Scenario: Minmus base recovered for funds. **Behavior:** Applies only to non-KSC routes whose Supply Run started docked to an origin depot. No vessels at origin → `Status = EndpointLost` . Vessels exist but empty → `Status = WaitingForResources` . Route persists. Resumes when resources appear. Surface origins have proximity fallback. KSC origins skip this check.

10.4 Destination tanks full

Scenario: Route delivers 200 LF. Base has 200/200 LF. **Behavior:** `Status = DestinationFull` . Cycle skipped -- no ghost replay, no origin deduction. Origin NOT deducted. Resumes when player uses fuel and capacity becomes available.

10.5 Destination partially full

Scenario: Base has room for 100 LF, full on Ox. Delivery: 150 LF + 183 Ox. **Behavior:** Per-resource independent: deliver 100 LF, 0 Ox. Origin pays the CostManifest / KSC dispatch cost (transit already happened). `ROUTE_DELIVERED` records requested vs actual amounts.

10.6 Player reverts past a dispatch

Scenario: Route dispatched at UT=50000. Player reverts to UT=49000. **Behavior:** Stock KSP revert/load restores route state from `.sfs` . Parsek timeline rewinds use the existing GameAction ledger path: route dispatch and delivery entries participate in epoch isolation and tombstone invalidation, so `EffectiveState.ComputeELS()` masks invalidated dispatches/deliveries and recomputes funds/resources without relying on save rollback alone.

10.7 Time warp past multiple cycles

Scenario: Three cycles due at UT=50000, 50500, 51000. **Behavior:** All processed sequentially. Each dispatch checks origin independently. First may deplete origin, blocking subsequent.

10.8 Transport still docked at recording end

Scenario: Player forgets to undock. **Behavior:** Validation fails. "Create Supply Route" absent. Tooltip: "Transport must undock from destination."

10.9 No cargo transferred during connection

Scenario: Player docks and undocks without transferring delivery cargo. **Behavior:** Validation fails. Tooltip: "No resource or inventory transfer detected."

10.10 Multiple connection windows in one recording chain

Scenario: Supply Run has dock-transfer-undock-dock-transfer-undock. **Behavior:** Validation fails for v1. UI explains that multiple delivery stops were detected and multi-stop route execution is deferred. The player can re-fly or split the mission into one-stop Supply Runs.

10.11 Route dispatch while player is at destination

Scenario: Player at Mun base when delivery arrives. **Behavior:** Destination loaded. Uses `Part.RequestResource()` . Resources appear in real-time.

10.12 Competing routes at same origin

Scenario: Two routes share Minmus base. Base has enough for one, not both. **Behavior:** v1 — FIFO by NextDispatchUT. Future: player-configurable priority.

10.13 Concurrent deliveries to same destination

Scenario: Two routes deliver to the same base on the same scheduler tick. The destination can accept one delivery but not both. **Behavior:** v1 processes route events in deterministic FIFO order: NextDispatchUT, then route Id as a stable tie-breaker. Each route recomputes capacity immediately before its own delivery. The first route may fill the destination, causing the later route to partially deliver or report undelivered cargo. v1 does not reserve shared destination capacity across routes.

10.14 Linked route partner paused (future)

Scenario: Route A and B linked. Player pauses B. **Behavior:** A dispatches on its own schedule. B resumes from its next scheduled dispatch when unpaused.

10.15 Source recording missing

Scenario: Source recording for a route is deleted or fails to load. **Behavior:** Status = MissingSourceRecording. Route cannot dispatch and cargo transfers stop. UI explains that the proof Supply Run is gone and the route must be recreated or the recording restored. If the recording is restored by loading/reverting to a save where it exists, or by restoring recording sidecars, route load/revalidation clears the status back to Active and recalculates NextDispatchUT.

10.16 Save/load round-trip

Scenario: Save, load. **Behavior:** All Route fields serialized in ParsekScenario OnSave/OnLoad. State restored exactly.

11. v1 Limitations

- **Capacity changes during transit:** Delivery clamps to maxAmount at delivery time. Excess is not delivered; ROUTE_DELIVERED records requested vs actual amounts. Origin was already deducted at dispatch.
- **Zero transit duration:** Dispatch and delivery may process in same frame. Acceptable.
- **EC-only delivery:** ElectricCharge remains excluded from route manifests as environmental noise, matching Phase 11 resource snapshot rules. EC-only Supply Runs are not route-eligible in v1.
- **Resource not on destination:** If delivery manifest includes a resource the destination has no tanks for, that resource is reported as undelivered, not silently skipped.
- **Concurrent endpoint production/consumption during route creation:** Delivery proof uses the conservative min(transportLoss, endpointGain) over the dock/undock window. If a third vessel, ISRU, converter, or drain changes the endpoint resource during that same window, route analysis may under-credit or reject the candidate rather than trying to attribute multiple simultaneous sources and sinks. v1 expects one meaningful cargo transfer per connection window.
- **Origin loaded vs unloaded:** Same loaded/unloaded distinction as delivery.
- **Scene handling:** Route scheduler runs in all scenes via ParsekScenario. FlightGlobals.Vessels available for endpoint resolution.
- **Revert mechanism:** Route state serialized in .sfs. Quicksave load restores Route ConfigNode. Timeline events use epoch isolation.
- **Inventory delivery:** Inventory items delivered to destination cargo slots. If destination lacks available slots or the part type doesn't fit, excess items are reported as undelivered.

- **v1 scheduler shape:** `CurrentCycleStartUT` , `CurrentSegmentIndex` , `PendingDeliveryUT` , and `PendingStopIndex` are serialized now for chain-sequential execution and future multi-stop compatibility. In v1 there is only one delivery stop; the fields are mostly visual sequencing plus the single delivery boundary, not a commitment to multi-stop route behavior.
- **Crew delivery:** Deferred. No generic kerbal generation in v1.
- **Route analysis edge cases:** The route analysis engine walks docking windows linearly. Complex patterns (dock to A, undock from A, dock to A again) are detected as separate candidate windows and rejected for v1. A window with no delivery cargo change is not route-eligible.

12. What Doesn't Change

- **Recording runtime behavior** — recording behavior is unchanged. The Recording schema is additively extended for logistics capture (§4.9): connection-scoped dock/undock manifests, stock connection target metadata, endpoint situation, and origin-dock proof. Logistics v1 reads those fields but never writes to recordings.
 - **Ghost playback engine** — no execution changes to `GhostPlaybackEngine`, `IPlaybackTrajectory`, or `IGhostPositioner`. The route scheduler uses the same playback engine as loops for visuals, but route state advances from UT-driven scheduler ticks rather than depending on playback-completed events.
 - **Loop system** — per-recording loop toggle, timing, cycle events all work as today. Routes do not use the loop system — they are siblings, not built on top of it. Both use the ghost playback engine, but through different scheduling paths.
 - **Chain system** — chain segments, dock/undock boundaries, snapshots all unchanged.
 - **Manifest capture systems** — `ExtractResourceManifest` , inventory manifests, crew manifests, connection-scoped dock/undock manifests, and connection-target capture exist on Recording as additive fields. Logistics v1 consumes resources/inventory and connection metadata read-only.
 - **Merge dialog** — route creation may be offered after commit/merge when analysis finds an eligible Supply Run, but the merge semantics themselves do not change.
 - **Crew reservation** — not touched in v1. Crew logistics is deferred until it can use named roster/crew-reservation semantics instead of generic kerbal generation.
 - **Game actions system** — route dispatch/delivery events are new event types in existing ledger. KSC-origin Career dispatch costs use existing funds-reservation checks. No changes to recalculation engine architecture.
 - **Map markers** — deferred. No map view integration in v1.
-

13. Module Architecture

The logistics route system follows the same module pattern as the game actions system (v0.6): a self-contained directory with a thin orchestrator connecting it to Parsek's lifecycle hooks. The route module is removable by deleting the directory and removing the orchestrator calls — no behavioral changes to recording, playback, or the game actions system.

13.1 Directory structure

```
Source/Parsek/Logistics/
  Route.cs           // data model (Route, RouteStop, RouteEndpoint, RouteStatus)
  RouteStore.cs      // static storage surviving scene changes (like
RecordingStore)
  RouteScheduler.cs  // dispatch/delivery evaluation + chain-sequential playback
(pure logic)
  RouteDelivery.cs   // resource and inventory mutation on loaded/unloaded vessels
```

```

RouteEndpointResolver.cs // vessel finding by PID + surface proximity fallback
RouteManifestComputer.cs // derive delivery/cost manifests from recording chain
resources/inventory
RouteAnalysisEngine.cs // chain walk + connection-window extraction from committed
recordings
RouteOrchestrator.cs // thin integration layer -- called from ParsekScenario hooks

```

13.2 Integration seams (5 total)

These are the only places where logistics code touches existing Parsek code:

Seam	Where	What
Save/Load	ParsekScenario.OnSave/OnLoad	RouteOrchestrator.OnSave(node)/OnLoad(node)
Scheduler tick	ParsekScenario.Update	RouteOrchestrator.Tick(currentUT)
Playback start/seek	ParsekPlaybackPolicy / ghost playback start path	RouteOrchestrator.RequestVisualPlayback(recording recordingLocalOffsetUT)
Playback completed	ParsekPlaybackPolicy.HandlePlaybackCompleted	RouteOrchestrator.OnVisualSegmentCompleted(evt)

Timeline events	Ledger / LedgerOrchestrator	New GameActionType entries for ROUTE_DISPATCHE / ROUTE_DELIVERED
------------------------	-----------------------------	--

No route-state changes to GhostPlaybackEngine , RecordingStore , RecordingTree , ChainSegmentManager , or RecordingOptimizer . Recording metadata changes are additive: logistics capture adds connection-scoped dock/undock manifests and origin-dock metadata for route analysis. The playback start/seek seam is visual only; if the existing ghost start path needs an offset parameter or adapter, that change must not make ghost playback authoritative for delivery. GhostPlaybackEngine continues to emit PlaybackCompleted at the same point it does today; only ParsekPlaybackPolicy / RouteOrchestrator interpret that event as a visual hint instead of an authoritative route-state transition.

13.3 Read-only consumption of recording data

The route module is a read-only consumer of recording data. It reads:

- rec.StartResources / rec.EndResources -- resource manifests (Phase 11)
- rec.StartInventory / rec.EndInventory -- inventory manifests (Phase 11)
- rec.TransportPartPersistentIds , rec.EndpointPartPersistentIds , and connection-scoped dock/undock resource/inventory manifests -- delivery proof while docked
- rec.TransferTargetVesselPid / rec.TransferKind -- stock connection target identification
- rec.TransferEndpointSituation and rec.StartDockedOriginVesselPid -- surface/orbit classification and non-KSC origin proof
- rec.StartBodyName , rec.StartLatitude , rec.StartLongitude -- location context (Phase 10)
- Chain boundary trajectory points for connection-time coordinates

The route module never writes to Recording objects. It creates Route objects in its own RouteStore , serialized in its own ROUTES ConfigNode section inside ParsekScenario .

13.4 Resource modification path

Resource delivery modifies vessels that are not part of the recording system — they are real KSP vessels at endpoint locations:

```
RouteDelivery.DeliverResources(endpointVessel, deliveryManifest)
    if loaded:    part.RequestResource(name, -amount)    // KSP API
    if unloaded:  protoPartResource.amount += amount     // direct field write
```

This is completely independent of Parsek's recording/playback/ghost systems. No ghost, no recording, no trajectory — just a vessel and a number.

13.5 Lifecycle isolation

Routes have their own lifecycle, independent of recordings:

- **Creation:** from a committed Supply Run after player confirmation, but the route is a separate entity with its own GUID.
- **Persistence:** own ConfigNode section (ROUTES in ParsekScenario), not part of recording metadata.
- **Deletion:** deleting a route does not affect its source recordings. Deleting a source recording disables the route (MissingSourceRecording); cargo transfers do not continue without the proof run.

- **Revert:** route state is serialized in .sfs — quicksave/load restores it. Timeline events use the existing epoch isolation. Loading/reverting to a save where a missing source recording exists again revalidates `MissingSourceRecording` routes and restores them to `Active` .

13.6 Why not a separate assembly now

The roadmap defers assembly extraction to the future standalone ghost-playback boundary. For Logistics / Supply Routes, a directory-level module within `Parsek.csproj` is the right granularity. Routes need direct access to `Recording.StartResources / EndResources` , `RecordingStore.CommittedRecordings` , `Ledger` , and `ParsekScenario` lifecycle. Cross-assembly access would require making all of these `public` or adding an interface layer — friction without benefit. If standalone ghost playback extraction happens, routes stay in Parsek (they are Parsek policy, not ghost playback).

14. Backward Compatibility

- **Saves without routes:** Load fine. `ROUTE` `ConfigNode` absent.
 - **Saves without resource manifests:** Load fine. Route creation unavailable for old recordings (no manifest data).
 - **Old recordings:** Cannot become routes. Player must re-fly to create a recording with manifests.
 - **Format:** All new data additive. No version bump. Missing nodes = no data.
 - **Reserved v1 fields:** v1 serializers preserve the forward-compatible route shape (`Stops` , `LinkedRouteId` , inventory manifests, `KscDispatchFundsCost` , `CurrentCycleStartUT` , pending transit fields). Non-null defaults are written; nullable fields use the omission-means-null convention from §4.8 and must restore as null. Skipping reserved fields turns future multi-stop/round-trip work into a save migration instead of an additive load.
-

15. Diagnostic Logging

15.1 Route creation

- `[Parsek][INFO][Route]` Route created: `id={id}` `name={name}` `recordings={count}` `endpoint={body}({lat},{lon})` `connection={kind}` `cost={manifest}` `kscFunds={funds}`
- `[Parsek][INFO][Route]` Route endpoint: `{body}({lat},{lon})` `vesselPid={pid}` `delivery={manifest}` `inventory={manifest}`
- `[Parsek][INFO][Route]` Route analysis: `chain={count}` `recordings`, `deliveryWindows={count}`, `acceptedStops={count}`
- `[Parsek][INFO][Route]` Route validation failed for chain: `{reason}`

15.2 Dispatch evaluation

- `[Parsek][VERBOSE][Route]` Dispatch check: `route={name}` `nextUT={ut}` `currentUT={ut}` `status={status}`
- `[Parsek][INFO][Route]` Dispatch: `route={name}` `cycle={n}` `deducted={amounts}` from origin at `{body}({lat},{lon})`
- `[Parsek][INFO][Route]` Dispatch: `route={name}` `cycle={n}` `chargedKscFunds={funds}`
- `[Parsek][INFO][Route]` Dispatch delayed: `route={name}` – origin missing `{resource}={needed}` (`available={have}`)
- `[Parsek][INFO][Route]` Dispatch delayed: `route={name}` – insufficient KSC funds needed=`{funds}` `available={funds}`
- `[Parsek][INFO][Route]` Dispatch skipped: `route={name}` – destination full (`capacity={amounts}`)

- [Parsek][WARN][Route] Dispatch disabled: route={name} – missing source recording id={id}
- [Parsek][WARN][Route] In-transit delivery aborted: route={name} – missing source recording id={id}

15.3 Delivery

- [Parsek][INFO][Route] Delivery: route={name} cycle={n} requested={amounts} actual={amounts} endpointPid={pid} at {body}({lat},{lon})
- [Parsek][INFO][Route] Partial delivery: route={name} – {resource} delivered {actual}/{requested}
- [Parsek][WARN][Route] Delivery failed: route={name} – no vessels at destination

15.4 Endpoint resolution

- [Parsek][VERBOSE][Route] Endpoint resolve: {type} pid={pid} found={bool} fallback={nearest-compatible/none} candidates={count}

15.5 State transitions

- [Parsek][INFO][Route] Status change: route={name} {old} → {new}

15.6 Timeline events

- [Parsek][INFO][Route] Timeline event: ROUTE_DISPATCHED route={name} ut={ut}
- [Parsek][INFO][Route] Timeline event: ROUTE_DELIVERED route={name} ut={ut} amounts={amounts}

16. Test Plan

16.1 Unit tests (pure logic, no Unity)

ExtractResourceManifest

- Empty ConfigNode → empty manifest. *Catches: null deref on missing PART nodes.*
- Single part, one resource → manifest with one entry. *Catches: parsing errors.*
- Multi-part, same resource → amounts summed. *Catches: overwrite instead of accumulate.*
- Multiple resource types → all present. *Catches: single-resource assumption.*
- Missing RESOURCE node → skipped gracefully. *Catches: null deref.*
- Zero-amount resource → included (maxAmount matters). *Catches: filtering out zeros.*

ComputeDeliveryManifest

- Normal transfer on connection-scoped part PID sets: transport 200->50 LF and endpoint 0->150 LF -> delivery 150 LF. *Catches: wrong snapshot pair.*
- Docked aggregate vessel unchanged while transport tank decreases and endpoint tank increases -> delivery detected from connection-scoped manifests. *Catches: merged-vessel false negative.*
- Transport tank decreases but endpoint does not retain the cargo -> no delivery manifest entry. *Catches: docked consumption masquerading as delivery.*
- Transport gains or endpoint loses a non-ignored resource/payload during the same window -> v1 candidate rejected. *Catches: pickup/mixed-transfer windows without algorithmic detection.*
- Stored part variant/resources differ -> inventory payload identities do not collapse into one part-name count. *Catches: fungible inventory cargo.*
- LiquidFuel increased across the connection window -> no v1 delivery, candidate rejected as pickup-only. *Catches: accidental pickup route.*

- Mixed delivery and pickup deltas after EC/IntakeAir filtering -> candidate rejected. *Catches: silently dropping pickup cargo.*
- EC and IntakeAir deltas -> ignored. *Catches: environmental noise route creation.*
- No resource/inventory delta -> empty manifest (validation rejects). *Catches: false positive.*

Route analysis engine

- Single dock-transfer-undock -> one stop extracted. *Catches: no stop found.*
- Two dock-transfer-undock delivery windows -> validation rejects v1 candidate. *Catches: charging full multi-stop run while delivering one stop.*
- Dock without undock -> validation fails. *Catches: incomplete pair.*
- No docking events -> validation fails. *Catches: false acceptance.*
- Docking window with no delivery resource/inventory change -> validation rejects. *Catches: empty do-nothing route.*
- Resource pickup window -> validation rejects. *Catches: accidental negative delivery.*
- Multi-body chain (Kerbin origin, Mun endpoint) -> correct body for endpoint. *Catches: assuming same body.*
- Non-DockingPort connection kind -> validation rejects in v1. *Catches: premature claw/fuel-line support.*
- Mun/Minmus landed endpoint -> `isSurface = true`. *Catches: airless-body altitude misclassification.*
- Non-KSC route without start-docked origin depot PID -> validation rejects. *Catches: unproven origin resource debit.*

Route validation

- Dock + delivery transfer + undock -> valid. *Catches: false rejection.*
- Dock + undock, no delivery transfer -> invalid. *Catches: false acceptance.*
- Dock + delivery transfer, no undock -> invalid. *Catches: missing undock check.*
- Non-docking connection -> invalid in v1. *Catches: unsupported producer accepted.*
- No docking connection -> invalid. *Catches: missing connection check.*
- Missing source recording -> route status becomes MissingSourceRecording. *Catches: cargo without proof recording.*
- Missing source recording restored on load/revalidation -> route returns to Active and recomputes NextDispatchUT. *Catches: unrecoverable missing-source status.*

Chain-sequential playback

- `CurrentCycleStartUT = null` and `CurrentSegmentIndex = -1` while idle; dispatch sets `CurrentCycleStartUT` and `CurrentSegmentIndex = 0`. *Catches: ambiguous idle/active state.*
- `CurrentSegmentIndex` indexes `RecordingIds`, not trajectory samples or `TrackSections`. *Catches: off-by-one against the wrong segment model.*
- UT reaches source recording 0 boundary -> scheduler starts source recording 1. *Catches: stuck on first recording.*
- UT reaches last source recording boundary -> cycle count incremented, status returns to Active and clears `CurrentCycleStartUT`. *Catches: stuck InTransit.*
- UT reaches stop boundary -> delivery triggered even without playback event. *Catches: missed delivery during warp/load/non-flight scenes.*
- Enter flight scene halfway through transit -> ghost seeks to scheduler elapsed offset or skips rendering, never restarts at recording UT 0. *Catches: rover-in-the-past visual rewind.*
- Playback completion event without due UT -> visual-only hint, no duplicate delivery. *Catches: event/UT double-processing.*
- Source recording deleted while InTransit -> status becomes MissingSourceRecording before delivery. *Catches: mid-transit cargo without proof recording.*
- Pause requested while InTransit -> delivery still executes, then route becomes Paused. *Catches: frozen in-flight cargo or accidental future dispatch.*

Dispatch evaluation

- KSC origin in Career, capacity available, funds affordable -> dispatch and funds spending emitted. *Catches: free KSC mass.*
- KSC origin in Career, funds insufficient -> `WaitingForFunds` , `NextDispatchUT` unchanged. *Catches: dispatch without funds or busy skip loop.*
- `WaitingForResources/WaitingForFunds` before `NextEligibilityCheckUT` -> no origin scan. *Catches: per-frame idle polling.*
- KSC origin in Science/Sandbox -> dispatch with no funds action and no `Funding.Instance` access. *Catches: career-only cost leaking or null Funding.*
- Non-KSC start-docked origin, sufficient resources/inventory -> dispatch, deducted from origin depot. *Catches: skipping deduction.*
- Non-KSC start-docked origin, insufficient -> delayed. *Catches: dispatching without resources.*
- Destination full -> skipped, origin NOT deducted. *Catches: wasted deduction.*
- Partial capacity -> per-resource independent, full route cost. *Catches: coupled delivery.*
- Non-KSC start-docked origin -> deducts from recorded origin depot, not transport or arbitrary nearby vessel. *Catches: wrong debit identity.*
- Two due deliveries to same destination -> deterministic FIFO (`NextDispatchUT` , then route id), second route sees capacity after first. *Catches: hidden nondeterministic contention.*
- `LinkedRouteId` set in v1 -> ignored by dispatch and no linked-partner log line is emitted. *Catches: accidentally enabling deferred round-trip behavior.*

Synodic period

- Kerbin-Mun → ~6.4 days. *Catches: formula error.*
- Mun-Laythe → walks to Kerbin/Jool and returns a positive Kerbin/Jool synodic period. *Catches: cross-system hierarchy.*
- Kerbin-Duna/Eeloo -> positive Sun-parent synodic period. *Catches: accidentally disabling interplanetary routes.*
- Same body → 0. *Catches: division by zero.*
- Origin or destination is the Sun → 0. *Catches: missing guard without breaking Sun-parent planets.*
- Equal periods → 0. *Catches: $T1 == T2$ division.*
- Inter-body route after pause/load -> next dispatch stays aligned to `DispatchWindowEpochUT` + $n * \text{DispatchWindowPeriod}$. *Catches: phase drift.*
- Player interval override -> increases minimum spacing without shifting phase. *Catches: window override misinterpreted as free phase shift.*

KSC cost

- Dry part cost excludes loaded resources and inventory contents. *Catches: double-counting cargo.*
- Resource cost uses `PartResourceDefinition.unitCost` * `CostManifest` amount . *Catches: free delivered resources.*
- Inventory delivered part cost includes `InventoryPayloadItem` dry part/module cost and stored resources from the captured `STOREDPART` snapshot. *Catches: free inventory parts.*

Endpoint resolution

- PID exists → found. *Catches: skipping PID.*
- KSC-origin route serializes `Origin.vesselPid = 0` and skips origin vessel resolution. *Catches: undefined KSC sentinel.*
- Non-KSC origin with `vesselPid = 0` -> invalid. *Catches: sentinel leaking into physical origins.*
- PID gone, surface, nearest compatible vessel within `SurfaceFallbackRadiusMeters` → fallback. *Catches: missing fallback.*

- PID gone, surface, nothing nearby → empty. *Catches: false match.*
- PID gone, orbital → empty. *Catches: orbital fallback.*
- Multiple surface vessels within fallback radius -> one nearest compatible endpoint, not aggregate warehouse. *Catches: magic-radius transfer.*
- Ghost map vessels, EVA, flags, debris, space objects, and vessels without compatible tanks/slots are excluded. *Catches: wrong fallback target.*

16.2 Log assertion tests

- Route creation logs manifest, connection kind, and endpoint. *Catches: silent creation.*
- Dispatch delayed logs missing resource and amounts. *Catches: silent delay.*
- Delivery logs requested vs actual amounts. *Catches: silent partial delivery.*
- Status change logs old→new. *Catches: silent transition.*
- Partial delivery logs reason. *Catches: silent partial.*
- MissingSourceRecording logs the missing source id. *Catches: disabled route with no diagnosis.*
- `LinkedRouteId` set in v1 does not log partner checks. *Catches: half-enabled round-trip scheduling.*

16.3 Serialization round-trip tests

- Route serialize → deserialize → all fields match. *Catches: missing field.*
- Stops list round-trip with one v1 stop. *Catches: endpoint lost.*
- RecordingIds list round-trip → all IDs preserved in order. *Catches: chain ordering lost.*
- ResourceManifest round-trip with full precision. *Catches: locale formatting.*
- InventoryPayloadItem manifests, stored STOREDPART snapshots, and KscDispatchFundsCost round-trip. *Catches: stock cargo/cost data lost.*
- Null LinkedRouteId survives round-trip. *Catches: empty vs null.*
- Null `CurrentCycleStartUT`, `PendingDeliveryUT`, and `NextEligibilityCheckUT` are omitted on save and restore as null. *Catches: sentinel/null mismatch.*
- In-transit route with `CurrentCycleStartUT`, `PendingDeliveryUT`, `PendingStopIndex`, and `CurrentSegmentIndex` survives. *Catches: transit state lost.*
- Waiting route with `NextEligibilityCheckUT` survives. *Catches: retry cadence lost.*
- In-transit route with `PauseAfterCurrentCycle` survives. *Catches: pause request lost across save/load.*
- In-transit route after save/load recomputes actual deliverable capacity at delivery time. *Catches: stale pending amount delivery.*

16.4 Integration tests (synthetic recordings)

- Recording chain with dock+delivery+undock -> route analysis extracts one endpoint. *Catches: chain structure mismatch.*
- Recording chain with two delivery docking windows -> validation rejects v1 route creation. *Catches: accidental multi-stop behavior.*
- Recording chain without undock -> validation rejects. *Catches: missing chain event check.*
- Inject route into save -> loads correctly with endpoint, manifests, KSC cost, and source ids. *Catches: ParsekScenario integration.*

16.5 In-game tests (KSP runtime)

These complement the pure-logic visual-handoff tests in §16.1: unit tests prove the scheduler chooses the right recording-local offset, while in-game tests prove the live ghost playback and loaded-vessel stock APIs honor that decision.

- Loaded-vessel resource delivery uses `Part.RequestResource()` and clamps to current tank capacity. *Catches: unloaded-only implementation passing unit tests.*

- Loaded-vessel non-KSC origin deduction removes resources from the proven origin depot. *Catches: dispatch cost mutation hitting the wrong vessel.*
 - Loaded `ModuleInventoryPart` delivery reconstructs exact `STOREDPART` payloads and respects slot limits. *Catches: inventory slot accounting only working in serialized `ConfigNodes`.*
 - Flight-scene entry during an in-transit cycle starts/seeks the route ghost at scheduler elapsed time. *Catches: visual replay from the beginning after scene change.*
-

17. Deferred Work

- **Record Supply Run helper:** v1 should automatically prompt after eligible committed runs. A helper button may be added later for intent marking or prompt filtering.
 - **Non-docking stock connection producers:** claw/grapple and stock crossfeed/fuel-line paths are deferred until docking routes are reliable. They need KSP API investigation for endpoint PID, connection start, connection end, and cargo delta.
 - **Pickup routes:** v1 is delivery-only. Resource and inventory pickup routes need separate stock-slot and part-identity tests before exposure.
 - **Non-KSC undocked-start origins:** v1 non-KSC routes require the Supply Run to start docked to a real origin depot. Common patterns where a tanker launches from a Minmus surface base, drives/flyes away undocked, and later docks to a destination are deferred until origin ownership can be proven without inventing a warehouse.
 - **KSC cost tuning:** v1 charges stock-realistic funds for source vessel parts plus used/delivered resources and inventory. This can be revisited later if repeated dispatch costs need recovery modeling.
 - **Force dispatch now:** v1 schedules dispatch from `NextDispatchUT` only. A later explicit "Dispatch now" UI action would need its own affordability/capacity checks and would set `CurrentCycleStartUT` from the action UT.
 - **Map view integration:** Route lines on the map. Deferred.
 - **Dispatch priority for competing routes:** v1 uses FIFO by `NextDispatchUT`.
-

Appendix A: Gameplay Scenarios (from Step 2)

A.1 Fuel Delivery Rover

Base with empty fuel tank near KSC. Rover drives to base, docks, transfers 150 LF through stock resource transfer, undocks, drives clear. Route: 150 LF per cycle, KSC origin with Career dispatch cost, interval = recording duration.

A.2 Orbital Monoprop Resupply

Kerbin station at 100km. Capsule from KSC: launch, dock, transfer 650 MP, undock, deorbit. Ghost visible during launch and station approach (RELATIVE frame). Transit invisible.

A.3 Minmus Ore Delivery (chained routes)

Route A: KSC → Minmus base (fuel). Route B: Minmus base → Kerbin depot (ore). B is gated by resource availability at Minmus. A feeds B.

A.4 Eeloo Supply Run (interplanetary)

Dispatch at Kerbin-Eeloo synodic period (~1.9 years), phase-anchored to the original Supply Run start UT. Gravity assists use the same two-body approximation. Player can increase spacing but v1 does not shift the phase anchor.

A.5 Failure Cases

Destination destroyed (surface: nearest compatible fallback; orbital: route pauses). Origin empty (delayed). Destination full (skipped, no deduction). Source recording missing (route disabled). Revert (epoch isolation). Time warp (sequential processing). Transport still docked (validation rejects).

Appendix B: Reference Documents

- `docs/dev/done/plans/phase-11-resource-snapshots.md` — archived Phase 11 plan (base resource snapshots; older route-recording workflow superseded here)
- `docs/dev/research/logistics-network-design.md` — logistics network research
- `docs/dev/research/loop-playback-and-logistics.md` — loop mechanics and orbital drift
- `docs/dev/research/resource-snapshots-preparation.md` — infrastructure analysis and unloaded vessel modification
- `docs/mods-references/` — background resource processing patterns from other KSP mods
- `docs/roadmap.md` — Phase 11, 11.5, 12